

ResearchHub · Technical Briefing

Research engine + QuickPlot — doc set for the frontend, backend, and BA teams. Generated from the survey-research codebase.



Table of Contents

Reading order for the ResearchHub teams (frontend / backend / business analyst), per `00-INDEX.md`.

- 00 ResearchHub — Technical Briefing (Index)
- 01 01 — System Overview (BA + everyone)
- 02 02 — Engine Deep Dive (Backend team)
- 03 03 — Frontend Deep Dive (Frontend team)
- 04 04 — Data Sources Registry (BA + backend team)
- 05 05 — Deployment & US Egress (DevOps + backend team)
- 06 06 — Glossary & Conventions (every team, quick reference)
- 07 07 — Opportunities & Roadmap (BA + leadership)

ResearchHub — Technical Briefing (Index)

Everything a new developer, a frontend engineer, a backend engineer, or the BA needs to understand about how the ResearchHub survey-research product actually works — from the smallest variable up to the deployment graph. Written from the code, with `file:line` citations so nothing is "trust me".

The three codebases (why there are three)

#	Repo	Path	What it is	Status
1	SurveyResearch (POC)	<code>poc01/SurveyResearch</code>	The original working prototype: a local FastAPI web app with two UIs — Classic (address-first tool) and QuickPlot (order-centric hub). Contains the battle-tested county registry and the data-source services (<code>parcel</code> , <code>fema</code> , <code>ngs</code> , <code>clerk</code> , <code>geocode</code> , ...).	Working; do not break its classic app. Its data + services are the crown jewels.
2	researchhub-engine	<code>researchhub-engine</code>	The standalone engine extracted from the POC: a production-shaped FastAPI service with a two-phase research pipeline, a Celery worker, its own tables (Postgres migrations), and a frozen <code>/api/v1/research/*</code> contract.	This is the forward-looking codebase. Most of this briefing is about it.
3	mapperty-reference (parent)	<code>mapperty-reference</code>	The larger monolith this product will live inside. Already deployed to AWS us-east-1 (ECS) , with Cognito auth. Its <code>orders / evidence / identity</code> modules are where the engine's stand-ins will be swapped in.	Parent owns auth, hosting, tenants.

The engine was deliberately built **standalone** (own `orders / tenants / files` stand-ins) so it can be tested and demoed on its own; at integration it drops onto the parent's real tables. That swap list is the "seams" table in every deep-dive.

The one-sentence product

A surveyor clicks an order, the system fetches/roots out the mortgage-survey evidence for a property: **Parcel ID, Deed, Plat, Flood map, survey control (NGS), and the appraiser tax record** — downloading what has a public data source and giving a verified one-click link for everything else. Nothing is a dead end.

Reading order by audience

BA / product owner	→ 01 System Overview → 04 Data Sources → 07 Opportunities
Frontend team	→ 01 System Overview → 03 Frontend Deep Dive → 06 Glossary
Backend team	→ 01 System Overview → 02 Engine Deep Dive → 04 Data Sources → 05 Deployment & Egress → 07 \$parent-integration
DevOps / hosting	→ 05 Deployment & Egress
Everyone	→ 06 Glossary (the vocabulary used everywhere)

Document index

Doc	Audience	Contents
00-INDEX.md	all	This file — repo map + how to read the set
01-system-overview.md	BA + all	The six survey documents, the two UIs, how a request flows end-to-end, the fallback philosophy, the Evidence Locker + locking rules, live coverage numbers
02-engine-deep-dive.md	Backend	Every module & function of the engine: contracts, schemas, ORM, service, router, worker, orchestration (the 11-adapter two-phase pipeline), storage, alembic chain, all seams
03-frontend-deep-dive.md	Frontend	Classic (index.html + src/*) and QuickPlot (quickplot.html + src/qp/*) file-by-file: load order, every component, state flow, the api() client, hash router, design tokens, every screen → its API calls, the embed-into-parent path
04-data-sources-registry.md	Backend/BA	The county registry structure + merge precedence, the state modules, the six data authorities with real sample queries, the coverage-verification gate, linkcheck verdicts
05-deployment-egress.md	Backend/DevOps	Dev run paths, the live E2E harness, the Docker stack, AWS us-east-1 hosting, the US-egress/VPN decision , the real remaining risk (shared-IP WAF/rate-limit blocking)
06-glossary-conventions.md	all	Every term/variable/convention: statuses, doc-type strings, org_id , lock_seal , situs , the buffered parcel query (75/150/300 m), build_key guard, "don't fix the TX URL", ...
07-opportunities-roadmap.md	BA + product	Committed backlog P1–P6 with order-volume math, the SaaS productionization table, brainstormed new ideas, and the parent-integration task checklist

Repo map (top level)

ResearchHub/	
└─ poc01/SurveyResearch/	# POC – 2 UIs, services, county registry
│ └─ backend/app/main.py	# FastAPI app (classic + /api/v2)
│ └─ backend/app/services/	# geocode · parcel · appraiser · clerk.scrapers · fema · ngs · downloader · jobs · evidence · storage · http
│ └─ backend/app/data/	# county_platforms.py registry · states/* · geography · reference · records_links.json
│ └─ backend/app/quickplot/	# v2 layer (orders/research/locker/catalog)
│ └─ frontend/	# index.html (Classic) + quickplot.html (QuickPlot)
└─ researchhub-engine/	# THE ENGINE
│ └─ backend/app/engine/	# contracts · schemas · models · service · router · worker · adapters · orchestration/
│ │ └─ orchestration/	# context.py · sources.py · steps.py · runner.py · folders.py
│ │ └─ order_source.py	# dev `orders` stand-in
│ │ └─ evidence_source.py	# dev `files`/`order_files` stand-in
│ └─ backend/alembic/	# 0002 → 0001 → 0003 → 0004 migration chain

```
|   ├── scripts/                #   export_contracts.py · e2e_local.py
|   └── tests/                  #   1097 offline tests (+ `‑m pg` integration tier)
└─ mapperty-reference/         #   PARENT – AWS us-east-1, Cognito, real orders/evidence/identity
    └─ app/modules/            #   orders (stub) · evidence (gap) · identity · survey · admin ...
```

Ground rules these docs assume

- **The POC's `result.json` vocabulary is frozen** — `ok` | `link` | `empty` | `error`. Engine-only reasoning rides in *additive* fields (`source_outcome`, `confidence`, `provenance`, `error`).
- **No URLs are invented.** Every records link came from a live-verified `.gov/.org/.us` source and is re-checkable via `/api/linkcheck`.
- **Address is the only required field.** Selected county wins over the geocoded one.
- File paths cited like `backend/app/engine/service.py:149` are relative to the repo in the header of each doc unless stated otherwise.

01 — System Overview (BA + everyone)

How the ResearchHub product works end to end, in plain language with diagrams. No code required to follow this; the deep-dives reference it.

1. The problem we automate

For a **residential mortgage / location survey**, a surveyor must first assemble the *research set* for the property — the public records that prove where the parcel is, what's recorded on it, whether it floods, and what survey control exists nearby. Today that is hours of clicking county websites. This product automates the *research phase*: enter the address (or parcel ID), get the evidence set.

2. The six documents (the "research set")

#	Document	What it is	Where it comes from	Auto-fetch?
1	Parcel ID	The assessor's parcel identifier + situs + lot area	County GIS (ArcGIS) or a statewide cadastral service	✅ where a queryable service exists (66.2% of US counties)
2	Appraiser / tax card	Owner, values, last sale, tax record	County property-appraiser portal	✅ wherever the parcel resolved
3	Deed	The recorded deed for the subject parcel	County clerk official records	⚠️ only 5 FL counties (Playwright scraper); else a verified deep-link
4	Plat	The subdivision map / recorded plat	County clerk official records	same as deed
5	Flood map + zone	FEMA NFHL flood zone (A/AE/X/...), FIRMette exhibit	FEMA National Flood Hazard Layer	✅ all counties (US egress required)
6	Survey control (NGS)	Nearby NGS benchmark datasheets (needed to tie the survey to state control)	NOAA National Geodetic Survey	✅ all counties

Four more **link-only** steps exist in the full survey document catalog (they have no public queryable source): **adjoiners**, **easements/ROW**, **prior surveys**, **condo declarations**, plus **BLM GLO** land-records as a link. These map to the source-registry checklist but are never auto-fetched.

"Nothing is ever a dead end." Every step carries a `link` fallback: either the source auto-fetches a file, or the user gets a verified one-click link straight to the right search screen of the right county portal (or, last resort, the NETROnline directory deep-link for that county).

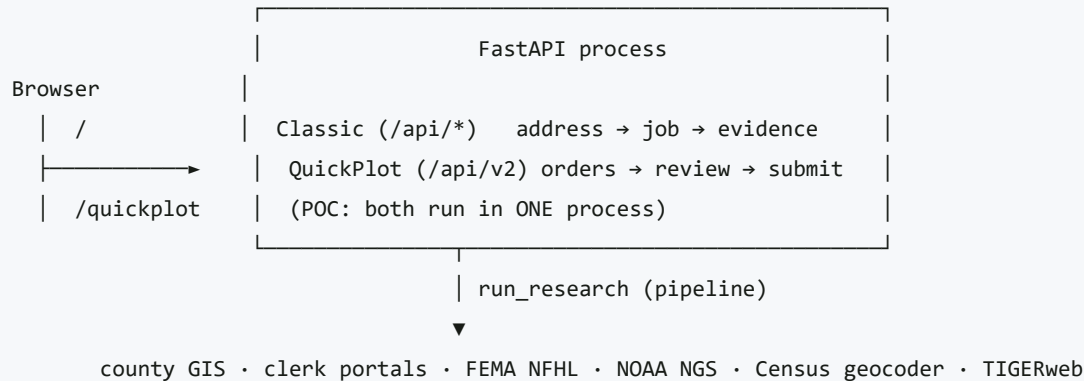
3. The four statuses the frontend understands

This vocabulary is **frozen** — the POC shipped it and it will not change meaning:

Status	Meaning	Example
ok	The engine fetched a real document	NGS datasheet downloaded
link	No auto-download; here is a verified link	"Open Clerk Official Records — Deed"
empty	Nothing exists / not applicable	No NGS marks within 3 km
error	The step failed (blocked / broken / timeout)	FEMA unreachable

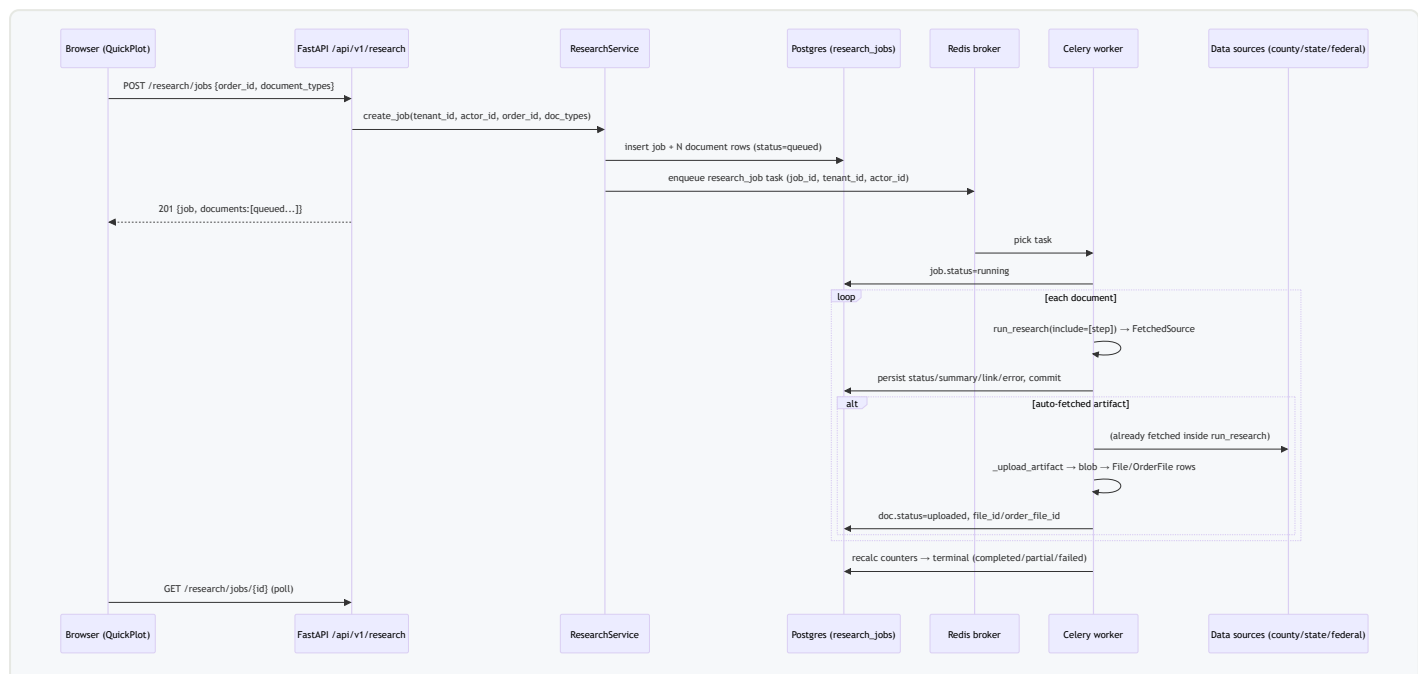
New engine-only reasoning (why a step is what it is) rides in **additive** fields next to these statuses — `source_outcome`, `confidence`, `provenance`, `error` — so the old consumers keep working untouched (`backend/app/engine/contracts.py:6`).

4. Two UIs today, one engine tomorrow



- **Classic** (/): the original address-first tool — paste an address, run the research pipeline, and get a job folder with the six documents and an Evidence Locker.
- **QuickPlot** (/quickplot): the Mapperty-styled, **order-centric** hub — Orders have a stage rail (created → placed → research → field_survey → cad_drafting → delivered), a right-hand *source registry rail* (parcel, appraiser, plat, deed, flood, ... each ticked when fetched), per-document **Review & Lock**, and an append-only change log. This is the slice that will live inside the parent product.
- **researchhub-engine** (standalone service): the next generation of the pipeline. It takes the same county services and adds a two-phase engine, a background **Celery worker**, its own `research_jobs` / `research_documents` tables, and a frozen `/api/v1/research/*` API. QuickPlot will call this instead of running the pipeline in-process.

5. The engine request flow (the important picture)



The request/response happens in milliseconds; all the fetching happens **in the background worker**. `POST /jobs` just validates + persists + enqueues. The UI polls `GET /jobs/{id}`.

6. The fallback philosophy (why "error" is rare)

The pipeline is built so a source is never allowed to crash a run:

1. **Adapters never fall back internally.** An adapter either returns data or raises `SourceError(outcome, code, message, retryable)`.

2. **The step builder catches everything.** Any exception is *classified* (`blocked` = 401/403/406/429 WAF/geo block; `broken` = 404/410/5xx broken URL; `retryable` = timeout/connection) and degraded to a **link step + structured ErrorInfo**.
3. **The job still completes.** A run where 5 of 6 sources returned links and 1 auto-fetched reaches `completed`; a run with ≥ 1 hard failure reaches `partial`, and the failed documents can be re-run via `POST /research/jobs/{id}/retry` (creates a fresh job for just those).

This is what the live E2E proves: from this sandbox's network, FEMA is blocked and the run still finishes `completed` with a FEMA link card.

7. From evidence to a defensible survey

The business value is an **evidence set you can defend**:

- Every auto-fetched file is stored as a blob under a tenant-scoped key, with a recorded `sha256` digest and a provenance record (which source, when, fallback chain).
- In QuickPlot, a document only **locks** after the reviewer confirms three things — the file is **legible**, it **matches the subject parcel**, and it's from the **source recorded** for the order. These three checks are recorded with the lock.
- Locked documents cannot be re-typed or deleted; deletes are **soft** so the audit trail always points at a real row; the checklist ticks on **locked**, not on fetched.
- Research cannot be submitted with **zero locked documents** (API returns 409).

The standalone engine keeps the evidence linkage too: each uploaded artifact becomes a `File` plus an `OrderFile` link row, so the blob ↔ order ↔ document chain is queryable.

8. Live coverage (as of the POC data — always check `/api/reference/coverage`)

Order-weighted, **FL + GA + TX = 92.6% of all orders**:

State	% orders	Parcel	Deed/Plat	Appraiser
FL	51.7%	67/67	67/67 (5 auto-download deeds)	67/67
GA	33.8%	90/159 (gap = polygon, paid API needed)	159/159	159/159
TX	7.1%	254/254	212/254	254/254
OH	1.5%	88/88	72/88	79/88
NJ	1.1%	21/21	21/21	21/21

Nationwide parcels: **2,082 / 3,144 counties (66.2%)**; 34 states have a statewide parcel service; every county has at least a NETROnline directory fallback.

9. Who runs where — the three-tier story for the BA

Tier	Repo	Runs on	For
POC Classic + QuickPlot	SurveyResearch	the user's Windows PC (dev) / office server (prod)	demo & current validation
Engine	researchhub-engine	Docker (db+redis+api+worker) anywhere; will join the parent on AWS	the productized pipeline
Parent	mapperty-reference	AWS us-east-1 (ECS) with Cognito	the real tenant-facing product

US egress (needed because FEMA & county portals geo-block non-US IPs) is answered fully in `05-deployment-egress.md`. Short version: **the parent is already on AWS us-east-1, so it's native — no VPN needed in production**; the POC's VPN was a developer-machine workaround.

Libraries & Versions (as of build)

The engine is a Python FastAPI service with the following key dependencies (pinned in `requirements.txt`):

Category	Package	Version
FastAPI framework	fastapi	0.111.0
	uvicorn	0.30.0
Data validation	pydantic	2.10.0
	pydantic-settings	2.3.4
Database & ORM	sqlalchemy	2.0.46
	alembic	1.13.2
	psycopg2-binary	2.9.10
HTTP client	httpx	0.28.1
	requests	2.32.5
Queue / worker	celery	5.4.0
	redis	5.0.7
S3 storage	boto3	1.43.88
Auth (Cognito JWT)	python-jose[cryptography]	>=3.3
Observability	structlog	>=24.0
Rate limiting	limits	>=3.11.0
Engine-only extras	Pillow	>=11.0 (flood-map compositing)
	playwright	>=1.40 (clerk deed/plat scraper)
Test only	pytest	>=8.0
	pytest-cov	>=5.0 (coverage floor 55%)
	moto	>=5.0 (S3 storage mock in CI)
	respx	>=0.21 (HTTP mock for callback tests)
	responses	>=0.25 (mock requests for adapter tests)

Next: `02-engine-deep-dive.md` (backend) · `03-frontend-deep-dive.md` (frontend) · `04-data-sources-registry.md` (data) · `05-deployment-egress.md` (hosting) · `06-glossary-conventions.md` (vocabulary) · `07-opportunities-roadmap.md` (what's next).

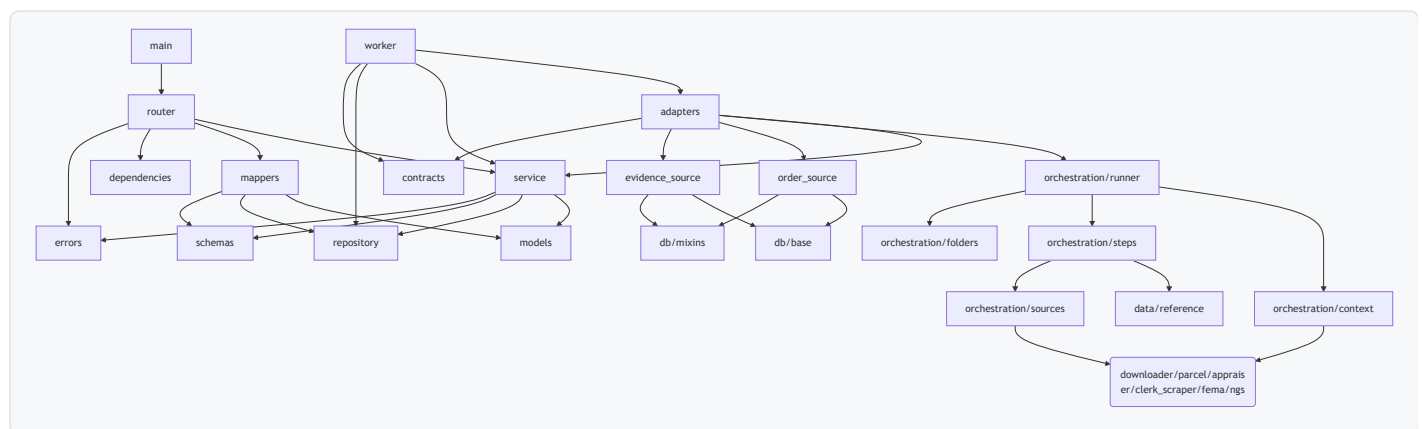
02 — Engine Deep Dive (Backend team)

The complete map of `researchhub-engine`. Every module, its functions, what they do, and how they link. All citations are relative to the repo root.

0. The shape

<code>backend/app/engine/</code>	
<code>contracts.py</code>	canonical vocabulary (enums + result models) – the single source of truth
<code>schemas.py</code>	<code>/api/v1/research/*</code> request/response models (the wire contract)
<code>models.py</code>	ORM: <code>research_jobs</code> + <code>research_documents</code>
<code>repository.py</code>	thin query layer (tenant-scoped)
<code>service.py</code>	<code>ResearchService</code> – job lifecycle business logic
<code>router.py</code>	the 5 HTTP endpoints
<code>dependencies.py</code>	tenant/actor deps (header placeholders)
<code>mappers.py</code>	ORM → contract models (incl. <code>FileReference</code>)
<code>adapters.py</code>	production wiring (<code>order_provider</code> , <code>document_fetcher</code> , <code>build_research_service</code>)
<code>worker.py</code>	Celery app + <code>research_job</code> task + <code>enqueue_research_job</code>
<code>order_source.py</code>	dev <code>`orders`/`tenants`</code> stand-in + <code>seed_dev_order</code>
<code>evidence_source.py</code>	dev <code>`files`/`order_files`</code> stand-in + <code>create_evidence/get_file_reference</code>
<code>errors.py</code>	exception hierarchy mapped to HTTP codes
<code>orchestration/</code>	
<code>context.py</code>	Phase A – <code>resolve_property_context</code> → <code>PropertyContext</code>
<code>sources.py</code>	<code>SourceAdapter</code> interface + 12 adapters + <code>ADAPTERS</code> registry
<code>steps.py</code>	<code>build_steps</code> – include-aware, per-step fallback + status/provenance
<code>runner.py</code>	<code>run_research</code> – wires Phase A + Phase B
<code>folders.py</code>	staging folder + <code>manifest.json/result.json</code> writers

Dependency (import) graph — who links to whom



Two rules keep this sane: **(1)** `contracts.py` is imported by everything and imports nothing of the app (the vocabulary must never import app code); **(2)** `adapters.py` is the only place that knows "which concrete services feed the pipeline" — the service layer only sees injected callables.

1. The vocabulary — `contracts.py`

The frozen + additive model. The six frontend-facing step statuses **never change meaning**:

Enum	Values	Notes
StepStatus (contracts.py:20)	ok link empty error	frozen — what the FE renders
SourceOutcome (contracts.py:28)	auto link_only blocked broken retryable manual_review	engine's <i>why</i> — replaces the old ad-hoc if/elif chains
Confidence (contracts.py:51)	high medium low none	does it actually pertain to the parcel?
JobState (contracts.py:59)	queued running completed partial failed reviewed archived	persisted job lifecycle
SourceState (contracts.py:79)	idle fetching fetched failed unavailable	the per-source UI rail states

Models: ProvenanceRecord (source_id, provider, url, retrieved_utc, attempt, fallback_chain, file, sha256), ErrorInfo (code, message, retryable), StepResult (the canonical per-step shape — 15+ frozen fields, 5 additive: source_outcome, confidence, provenance, warnings, error), ResearchResult (top-level payload — frozen fields match the POC result.json exactly, additive ones like completed_utc / job_state / docs_dir / survey_type ride alongside) (contracts.py:95-213).

2. The wire contract — schemas.py

Enums: ResearchJobStatus (queued running completed partial failed cancelling cancelled reviewed archived), ResearchDocStatus (queued fetching fetched uploading uploaded failed skipped), ResearchErrorCode (client 4xx / per-document / system 5xx — full list at schemas.py:81).

Requests: CreateResearchJobRequest {order_id, document_types[], idempotency_key?} (schemas.py:121), RetryResearchJobRequest {document_types?} (:145), CancelJobRequest {reason?} (:160).

Responses: FileReference {file_id, order_file_id, key, filename, size_bytes, sha256, mime_type} (:178), ResearchDocument (per-doc: status, source_outcome, confidence, file, summary, link, provenance, errors) (:190), ResearchJob (the poll target) (:241), ResearchJobSummary (list form) (:280). Everything is wrapped in the parent-style envelope: success {data: ...}, error {error: {code, message, details?}} (:298).

These are **exported** to contracts/openapi.json + contracts/schemas/*.json by scripts/export_contracts.py and treated as frozen — the only way to change them is to change the code, re-run the export, and show a clean diff.

3. The tables — models.py

research_jobs (one row per run)

tenant_id (indexed, isolation key), order_id (indexed), idempotency_key (unique per tenant), requested_doc_types (JSONB), status (SQL enum), denormalized counters total/fetched/uploaded/failed/cancelled_documents, started_at / completed_at, top-level error_code / error_message, optional callback_url, audit/timestamp/soft-delete mixins (models.py:50-201). Unique (tenant_id, idempotency_key) .

Why denormalized counters? So the jobs list page can render without counting child rows on every poll. recalc_job_counters keeps them in sync (service.py:86).

research_documents (one row per requested doc type)

job_id → research_jobs.id (CASCADE, kept as a real FK), order_id (denormalized), doc_type, status, source_outcome / confidence (engine-only), file_id / order_file_id (set after upload — plain UUID columns *here*; **FKs are restored at parent integration**), summary, link, link_label, provenance (JSONB), warnings (JSONB), error_code / error_message, retryable, retry_count, fetched_at / uploaded_at (models.py:204-377). Unique (job_id, doc_type) .

The base tables and the alembic chain

0002_dev_base_tables.py	(down_revision=None)	tenants · orders · files · order_files	← dev shim
0001_research_tables.py	(down_revision=0002)	research_jobs · research_documents	← FKs onto the shim
0003_cancel_reason.py	(down_revision=0001)	research_jobs.cancel_reason	← cancel `reason`
0004_callback_delivered.py	(down_revision=0003)	research_jobs.callback_delivered	← callback-delivery flag

0002 comes **first** because 0001's `order_id` → `orders.id`, `tenant_id` → `tenants.id`, `file_id` → `files.id`, `order_file_id` → `order_files.id`. FKs need those tables to exist on a fresh database. At parent integration: **drop 0002**, retarget `0001.down_revision` onto the parent's chain, and the FKs point at the parent's real tables. Run `alembic upgrade head` from the **repo root** (`alembic.ini` → `script_location = backend/alembic`). 0003 adds the optional cancel `reason`; 0004 records whether a callback was delivered so the worker can back off without hammering the callback URL.

Dev quick-path: `RUN_ENV=local` triggers `Base.metadata.create_all` at startup instead (`backend/app/main.py`), so SQLite/laptop development never needs a migration run.

4. The repository — repository.py

```
ResearchJobRepository: get(db, job_id, tenant_id) (tenant + soft-delete scoped), get_by_idempotency,
list_for_order (newest first), create, save. ResearchDocumentRepository: get, list_for_job, list_failed, create,
save.
```

Nothing clever; the cleverness is that **every query is tenant-scoped** — a caller can never see another tenant's rows even if a bug slips in upstream.

5. The service — service.py

The two domain dataclasses

- `OrderData` (`service.py:40`): the **minimal** order interface the engine needs — `id`, `address_line_1`, `city`, `state`, `county`, `parcel_id`, `lat/lon`, `survey_type`. During integration an adapter builds this from `app/modules/orders`; the stand-alone `order_source` builds it from the dev `orders` table.
- `FetchedImage` (`service.py:60`): the result of fetching one doc type — `doc_type`, `status`, `summary/link/link_label`, and the evidence linkage fields (`file_key`, `file_size`, `sha256`, `file_id`, `order_file_id`) plus `provenance/warnings/error` fields.

ResearchService — the whole job lifecycle

Constructor takes **four injected dependencies** (`service.py:125`), which is how tests run fully offline:

Dependency	Signature	Production wiring
order_provider	<code>(order_id, tenant_id) → OrderData</code>	<code>None`</code>
document_fetcher	<code>(order, doc_types[], tenant_id?) → dict[str, FetchedImage]</code>	<code>adapters.build_document_fetcher()</code>
file_storage	object with <code>save_file(...)</code>	kept for interface completeness (the fetcher path uses <code>services.storage</code> directly)
enqueue	<code>(job_id, tenant_id, actor_id)</code>	<code>worker.enqueue_research_job</code> ; <code>None</code> = skip (tests/standalone)

Public methods:

- `create_job` (:149) — resolve order → validate doc-types → idempotency check → create job + document rows → **commit** → enqueue. Returns the job. The commit-before-enqueue ordering is what guarantees the worker always finds a committed row.
- `get_job` (:182), `list_order_jobs` (:188).
- `retry_job` (:193) — only terminal jobs; collects failed doc types (or a caller subset); **creates a NEW job** (the original is never mutated — the audit trail stays intact).
- `cancel_job` (:224) — only `queued` / `running`; sets `cancelled`. (In-flight documents finish their current step; queued ones become `skipped` in the worker.)
- `_resolve_order` (:238) — 404 `OrderNotFoundError` if the order is absent, 422 `OrderNotResearchableError` if it has no address.
- `_validate_doc_types` (:248) — **normalizes legacy aliases** ("flood" → `FEMA_FLOOD_ZONE_FIRM`, any casing; step short-names → canonical), dedups preserving first-occurrence order, rejects unknowns with `InvalidDocTypesError`. This accepts what the client contracts actually send.

Module-level helpers: `recalc_job_counters` (:86) and `resolve_job_terminal_status` (:99) — the "all ok → completed / some failed → partial / none ok → failed" logic.

6. The HTTP surface — `router.py`

Five endpoints under `prefix="/api/v1/research"` (`router.py`:39):

Method/path	Function	What it does
POST <code>/research/jobs</code>	<code>create_research_job</code> (:71)	create + enqueue; takes <code>X-Idempotency-Key</code> header
GET <code>/research/jobs/{job_id}</code>	<code>get_research_job</code> (:104)	the poll target
POST <code>/research/jobs/{job_id}/retry</code>	<code>retry_research_job</code> (:131)	new job for failed docs
POST <code>/research/jobs/{job_id}/cancel</code>	<code>cancel_research_job</code> (:162)	cancel running/queued
GET <code>/research/orders/{order_id}/jobs</code>	<code>list_order_jobs</code> (:183)	history for an order

Every handler reads `actor_id/tenant_id` from the dependency overrides and calls `get_service` → `adapters.build_research_service()` (:46). **This single shared construction is what fixed the "worker built a service with None dependencies" bug** — the API and the worker literally call the same function.

Auth note: the docstring at `router.py`:15 says the current header-based deps are a *placeholder* to be replaced by Cognito wiring at integration. The endpoint signatures don't change.

7. The worker — `worker.py`

Celery app (:30) configured with `task_acks_late=True` + `worker_prefetch_multiplier=1` (a worker crash **requeues** the task rather than dropping it), JSON serialization, and a dedicated `research` queue.

`enqueue_research_job` (:46) — called by `create_job`; coerces UUIDs and publishes `research_job.delay(...)`.

`research_job` task (:61) — the only place the orchestrator runs. Step by step:

1. `build_research_service(); set running + started_at` (:67-78).
2. Pull documents; fetch the order via `order_provider` (:80-81).
3. Per document (:83-120):
 - `cancelled` job → mark `skipped`, continue;
 - mark `fetching`, call `document_fetcher(order, [doc.doc_type], tenant_id=tenant_id)`;
 - if `uploaded` + `file_key` → persist `file_id/order_file_id/uploaded_at`;

- if `failed` → persist `error_code` / `error_message` / `retryable`, bump `retry_count`;
 - any uncaught exception → `INTERNAL_ERROR`, `retryable`, `retry_count += 1`;
 - each document commits independently so one bad doc can't lose the rest.
4. `recalc_job_counters` + `resolve_job_terminal_status` + `completed_at` (:122-126).
 5. Optional `callback_url` POST (non-fatal on failure) (:128-129).
 6. Top-level exception → rollback + `self.retry(countdown=30) (max_retries=1) ( :131-133 )`.

Windows note: run the worker with `--pool=solo` — billiard's process pool raises `WinError 6/5` on Windows. Linux prod workers use the default prefork pool.

8. The pipeline — orchestration/ (the heart)

Phase A — `context.py`: resolve the property context once

`resolve_property_context(...)` (`job_number`, `address`, `survey_type`, `selected_state/fips`, `order_number`, `search_parcel_id` → `PropertyContext`):

1. **Parcel-ID pre-resolve** — if the caller supplied a parcel ID, resolve it directly (no geocoding needed).
2. **Geocode + county normalization** — Census geocoder (with ArcGIS + Nominatim fallbacks); county FIPS normalization; appraiser URL picked from the registry (`STATE_APPRAISER` → county → NETOnline).
3. **Parcel resolution** — county `gis_rest` → `STATE_PARCEL` → fallback. **Trust only on a positive address match**; a buffered/neighbor-only match is suppressed and tagged with a warning so the parcel never silently "finds" the wrong polygon.
4. **Run identity + clerk references** — job folder name, clerk official-records search URLs.

`PropertyContext` (dataclass) carries `parcel`, `parcel_ok`, `parcel_id`, `parcel_hints`, `appraiser_url`, `clerk_ref`, `folder`, `warnings`, plus `log` / `log_json` / `log_file` helpers and a `WarningBag` (dedup, ordered). `situs` and `land_sqft` / `land_acres` are properties the parcel step reads.

Phase B — `sources.py`: the 12 adapters

The interface that replaced the monolith's if/elif chain:

```
class SourceAdapter:
    key: str = "" # e.g. "parcel"
    def fetch(self, ctx, docs_dir) -> FetchedSource:
        raise NotImplementedError # override per source
    def fallback(self, ctx, error: ErrorInfo | None = None) -> FetchedSource:
        return FetchedSource(link="", link_label="", summary="")
```

`FetchedSource` (`sources.py:50`) — canonical adapter output: `status` (ok/link/empty), `data`, `summary`, `link` / `link_label`, `downloaded[]`, `saved_file`, `records[]` (manifest), `warnings`, `confidence`, and the parcel-only `situs` / `land_sqft` / `land_acres` / `address_match`.

`SourceError(outcome, code, message, retryable)` (:33) — the ONLY thing adapters raise. `classify_exception` (:91) maps arbitrary exceptions to `(outcome, code, message, retryable)`:

HTTP / exception	outcome	code
401/403/406/429	blocked	HTTP_xxx (WAF/geo — opens in a browser)
404/410	broken	HTTP_xxx (URL wrong/moved)
other 5xx	broken	HTTP_xxx (server error)
Timeout / ConnectionError	retryable	CONNECTION_FAILED

HTTP / exception	outcome	code
anything else	retryable	ADAPTER_ERROR

The registry (`ADAPTERS`, :625) — **12 entries**, 6 auto-fetch + 4 attempt-fetch + 2 link-only:

key	Adapter	Auto-fetch logic
parcel	ParcelAdapter (:114)	ok only if <code>arcgis-rest</code> + <code>parcels</code> + not buffered; else <code>PARCEL_NOT_FOUND</code> link. Saves <code>parcel.json</code> , downloads the record card if able
appraiser	AppraiserAdapter (:184)	ok if parcel resolved; downloads the county record card (Polk)
deed	DeedAdapter (:350)	ok if the clerk scrape returned a deed; confidence <code>medium</code>
plat	PlatAdapter (:396)	ok if the clerk scrape returned a plat
adjoiners / easements / prior_survey / condo	<code>_attempt_clerk_adapter(...)</code> (:438)	attempt the shared clerk scrape; fall back to a clerk official-records link
flood	FloodAdapter (:477)	FEMA NFHL zone; ok if zone or the composited map exhibit exists (Pillow)
benchmarks	NgsAdapter (:537)	NOAA NGS radial; ok if marks found (downloads up to 3 datasheets), <code>empty</code> if none
glo	GloAdapter (:580)	always link to BLM GLO
zoning	ZoningAdapter (:597)	always link to county zoning/GIS portal (no single auto-fetch source)

Shared clutch: `_scrape_documents` (:298) runs the **county-clerk Playwright scrape once per context** (cached on `ctx._clerk_docs`) so the deed and plat steps only ever open the browser once; the plat-book/page and OR book/page cues (`_plat_ref`, `_or_ref`, `_numref`, `_subdivision`) are ported verbatim from the POC orchestrator. `register_source(key, adapter)` (:643) is the test/extension seam.

steps.py: include-aware assembly + the fallback decision point

`build_steps(ctx, include=None)` iterates `RESIDENTIAL_DOCS` from `data/reference.py` in declared order, skipping keys not in `include`. For each: `_run_one` calls the adapter's `fetch`; on `SourceError` it calls `adapter.fallback` and attaches a structured `ErrorInfo`; on anything else it `classify_exceptions` and does the same. **A missing adapter maps to a link-only step — the runner never crashes.** `_assemble` then builds the canonical `StepResult` (summary, status, links, provenance made from the manifest entries, warnings) and logs the manifest.

`include` gates **Phase B only**. Phase A (context) always runs because phase B needs the context even for a single deed. A deed-only retry still geocodes → it just doesn't re-run the FEMA/NGS/parcel *document steps*.

runner.py: run_research(...)

Takes (`job_number`, `address`, `survey_type`, `include`, `selected_state`, `selected_county_fips`, `order_number`, `search_parcel_id`) → calls Phase A then Phase B, builds `ResearchResult` (`job_state=completed`, `completed_utc`, `docs_dir`), writes `manifest.json` + `result.json` into the staging folder. It contains **zero per-source branching** — that lives entirely in steps/sources.

folders.py

`job_folder(...)` → `JOBS_DIR/<slug>/research` staging; `save_json`, `write_manifest`, `write_result`. Scratch only — blob paths are never built here.

9. The adapters — production wiring

`adapters.py` holds the maps between orchestrator step keys and engine `DocumentType` S: `STEP_TO_DOC_TYPE` (`parcel` → `PARCEL_RECORD`, `appraiser` → `PROPERTY_APPRAISER_TAX_RECORD`, `plat` → `RECORDED_PLAT_SUBDIVISION_MAP`, `deed` → `DEED_SUBJECT_PARCEL`, `flood` → `FEMA_FLOOD_ZONE_FIRM`, `benchmarks` → `NGS_CONTROL`), plus the inverse `DOC_TYPE_TO_STEP` (:49). These 6 are the **only** types the API can request (`CreateResearchJobRequest.document_types`); an empty list resolves to all 6. The remaining reference steps (adjoiners, easements, prior_survey, condo, glo, zoning) are reachable only via a direct `run_research()` all-docs run — see `docs/regression-matrix.md` §2b.

- `order_provider(order_id, tenant_id)` (:51) — delegates to `order_source.order_provider`.
- `build_research_service()` (:61) — the single assembly point: `ResearchService( order_provider=order_provider, document_fetcher=build_document_fetcher(), enqueue=worker.enqueue_research_job)`.
- `build_document_fetcher()` (:76) — returns `fetch(order, doc_types, tenant_id=None)`: maps doc types → step keys, runs `run_research(include=steps, ...)`, and for each step builds a `FetchedException`; ok steps call `_upload_artifact`.
- `_upload_artifact(doc, downloaded, docs_dir, order_id, doc_type, storage, build_key, tenant_id)` (:135) — copies the first staged artifact into the blob store, hashes it, creates `File` + `OrderFile` rows (via `_create_evidence_rows` → `evidence_source.create_evidence`), and stamps `file_key / file_name / file_size / sha256 / file_id / order_file_id` on the `FetchedException`. This is the fix for the worker `INTERNAL_ERROR` ('FetchedException' object has no attribute 'file_id') — the evidence linkage is now complete end to end.
- `_map_step_status` (:206) — `ok` → `uploaded`, `link` → `fetching`, `empty` → `skipped`, `error` → `failed`.
- `_error_code_for` (:216) — `outcome/code` → `ResearchErrorCode` (`SOURCE_UNAVAILABLE`, `TIMEOUT`, `PARCEL_NOT_FOUND`, ...).

10. The stand-ins — `order_source.py` / `evidence_source.py`

Because the parent's `orders` and `evidence` modules are not wired yet, the engine ships self-contained stand-ins mirroring the parent's column names:

- `Order` (`orders`) + `Tenant` (`tenants`); `order_provider(order_id, tenant_id)` reads them; `seed_dev_order(...)` creates an idempotent tenant+order (used by the E2E).
- `File` (`files`: `filename`, `content_key`, `mime_type`, `size`, `sha256`) + `OrderFile` (`order_files`: `order_id`, `file_id`); `create_evidence(...)` → (`file_id`, `order_file_id`) and `get_file_reference(file_id)` → `dict` | `None`.

Swap points at integration (documented in each module docstring): replace `order_provider`'s body with a reader over `app/modules/orders`; and make `_to_file_reference` (`mappers.py`:23) read the parent's evidence `File` row. The call signatures — and everything above them — do not change.

11. The mappers — `mappers.py`

The single place ORM rows become contract models. `_to_file_reference` (:23) builds a `FileReference` from stored evidence metadata (real key/size/sha), `_s3_key` (:48) is the fallback key shape, `_to_document_schema` (:53) and `_to_job_schema` (:74) build the response objects, deliberately hiding `callback_url` and `idempotency_key` from consumers.

12. Errors — `errors.py`

`ResearchEngineError(code, http_status, message)` base + subclasses mapped to HTTP codes:

`OrderNotFoundError` (404), `OrderNotResearchableError` (422), `InvalidDocTypesError` (422), `JobNotFoundError` (404), `JobNotRetryableError` / `JobNotCancellableError` (409), `IdempotencyConflictError` (409), `MissingAddressError` (422), `DocumentFetchError` (200-ish per-doc). `ERROR_MAP` drives the exception handlers registered in `backend/app/main.py`.

13. Storage — backend/app/services/storage.py (blob abstraction)

The engine never builds blob paths itself; it calls:

- `build_key(org_id, order_id, document_id, filename)` — **raises if document_id is falsy**. This is the guard against the "None.pdf" regression where every document in an order silently shared one blob.
- `store = LocalStorage | S3Storage` behind one interface (`put/get/open/delete/exists/local_path`, plus `copy_in` used by the pipeline). Selection: `QP_STORAGE_BACKEND=local|s3`; a broken S3 config falls back to local with a logged line — **bad configuration never takes the app down**.
- `MAX_UPLOAD_BYTES` (default 50 MB).

Keys are tenant-scoped: `{tenant_id}/{order_id}/{doc_type}<ext>`.

14. Contract discipline & the tests (1097, all green)

- `scripts/export_contracts.py` re-exports `contracts/openapi.json` + `contracts/schemas/*.json` and exits 1 if no paths were written — CI-checks contract drift.
- `tests/conftest.py` sets env *before* app import (`DATABASE_URL=sqlite:///memory:`, `RUN_ENV=test`, `CELERY_TASK_ALWAYS_EAGER=true`) and patches SQLite's JSONB compiler so the Postgres-flavored models run in-memory. Fixtures: `engine`, `test_db` (fresh per test — imports `models/order_source/evidence_source` to register tables), `client` (overrides `get_db/auth`).
- The suite: **1097 tests collected** (up from 202), ~3 s, no network, plus an **integration-postgres** tier (`pytest tests -m pg` → `tests/test_pg_migrations.py`) that runs the real `0001...0004` alembic chain against a live Postgres in CI.
- Extra tiers added with the hardening pass: contract-drift, data-registry, negative-inputs, callback delivery, S3 storage (moto), clerk-scraper parsing (resp/respnses), and source-adapter classification — see `docs/validation-plan.md` for the mapping.
- Static gates are enforced in CI:** `ruff` (backend + tests), `mypy` (99 source files, `[tool.mypy]` in `pyproject.toml`), and `pytest --cov=app --cov=backend --cov-fail-under=55` (measured ~62%, above the 55% floor).
- The regression files pin the 11-step order, the frozen vocabulary, and every stable field — a schema change that breaks them is a contract break by definition.

15. All the seams a backend dev will touch

Seam	Where	Swap for
<code>order_provider</code>	<code>adapters.py:51</code> → <code>order_source.py</code>	parent <code>app/modules/orders</code> reader
<code>File / OrderFile</code>	<code>evidence_source.py</code>	parent evidence module
tenant/actor deps	<code>dependencies.py</code>	Cognito <code>app/modules/identity</code> deps (same signatures)
alembic <code>0002</code> shim	<code>0002_dev_base_tables.py</code>	drop; retarget <code>0001.down_revision</code>
<code>doc_types</code> vs source registry	<code>service._validate_doc_types</code>	validate against the live catalog at startup (TODO remains)
<code>file_storage</code> arg	<code>service.__init__</code>	used by evidence module path at integration
Windows <code>--pool=solo</code> worker	E2E/scripts	Linux default prefork in prod

16. Known hardenings (honest list)

- No real-world `error`-status fixture was ever captured (none of the 7 POC fixtures emitted one) — the classification logic is unit-tested (see `tests/test_source_adapters.py` + `test_api_negative_inputs.py`), but a captured production `error` fixture would let regressions fail loudly.

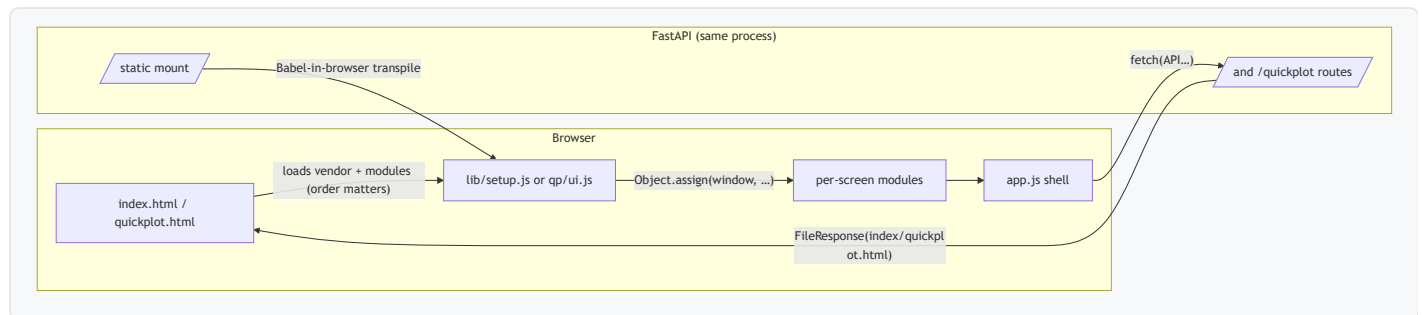
- S3 storage backend is smoke-tested against `moto` (`tests/test_storage_s3.py`, no AWS creds), but not against a real bucket here.
- Redis distribution through multiple uvicorn workers is exercised only by the single-worker E2E; the seam for scale-out is `worker.py` (`enqueue_research_job` / the task itself) and nothing else.

03 — Frontend Deep Dive (Frontend team)

The complete map of the two UIs: **Classic** (the address-first tool at `/`) and **QuickPlot** (the Mapperty-styled order-centric research hub at `/quickplot`). Both are single-file React apps with **no build step** — Babel transpiles in the browser, React + Babel are served locally from `frontend/vendor/`, and both talk to the same FastAPI process.

Physical location today: `poc01/SurveyResearch/frontend/` (alongside the POC backend). At integration, these files move into the parent repo unchanged — the static file mount (`StaticFiles` on `/static`) and the two routes below are what make that work without a build.

0. No-build anatomy (both apps)



Rules that keep it working:

- `<script type="text/babel">` runs in **document order** — the "load order matters" comments in both HTML files are load-bearing (`frontend/index.html:19` comment; `frontend/quickplot.html:19`).
- Modules can't `import`; every file ends with `Object.assign(window, {...})` publishing its declarations (see `frontend/src/app.js:73`, `frontend/src/qp/ui.js:298`).
- JSX must stay **valid XML** — a stray unclosed tag renders the whole page blank (Babel stops mid-transpile), so edits arrive as balanced tags, and preview on a hard refresh.
- No-cache is enforced server-side (`backend/app/main.py:35` `_revalidate_frontend` sets `Cache-Control: no-cache, must-revalidate` on `/`, `/quickplot`, and `/static/*`) — an edit shows on a plain refresh.

1. Classic (address-first) — `/`

The shell — `frontend/index.html` + `frontend/src/app.js`

`index.html` loads four vendor libs (React production, ReactDOM production, Babel, Tailwind — all tiny local copies, **no CDN**), then the modules in dependency order: `lib/setup.js` → `primitives.js` → `documents.js` → `DocViewer.js` → `Row.js` → `Results.js` → `search.js` → `jobs.js` → `about.js` → `locker.js` → `app.js` (`frontend/index.html:19-29`).

`App()` (`frontend/src/app.js:3`) owns one state blob `search` — `{mode, addr, pid, st, result, loading, error}` — and a top nav of four views: **Search / Jobs / Evidence Locker / About** (`app.js:7-8`). The left rail is a fixed `w-60` sidebar with the G-Source gradient (`#242a63` → `#2c357e`) and `#33409E` accent.

- `runSearch` (`app.js:13`) supports two modes: `address` (`POST {address}` to `/api/research`) and `parcel` (`POST {parcel_id, state}`). An `AbortController` is kept in `abortRef` so `cancelSearch` can abort the in-flight fetch mid-run (`app.js:25-37`).
- The content area renders the current view: `Search`, `Jobs`, `EvidenceLocker`, `About` (`app.js:59-62`).

The shared toolbox — `frontend/src/lib/setup.js`

The classic app's global utilities:

- `API = ""` — same-origin, so the app works unchanged behind the FastAPI process (`setup.js:4`).

- `icons.*` — 20+ inline SVG paths (search, folder, locker, droplet, target, ...) (`setup.js:11-41`) plus `docIcon(key)` mapping document keys (parcel/appraiser/deed/plat/...) to icons (:50).
- `viewableType(fn) → pdf | image | html | other` for the in-app viewer (:43).
- `jobFileUrl(jobName, fn, download) → /api/jobs/file?name=...&file=documents/<fn>` (:44).
- `aerialUrl(lat, lon) → free keyless Esri World Imagery` static export (Web-Mercator bbox around the lot) used as the aerial thumbnail on search results (:46-48).
- `attrGet(attrs, ...needles)` — case-insensitive situs-field picker used to compose the parcel address from a CAMA/parcel payload (:49).

The screens

File	Component(s)	What it does
<code>components/search.js</code>	<code>Search</code> , <code>Loader</code> , <code>Fact</code>	Address/parcel form, state+county+city dropdowns (counties + cities + parcel-states are fetched from <code>/api/reference/*</code>), a big elapsed-timer loader while <code>/api/research</code> runs, verdict + notes hooks after results return
<code>components/Results.js</code>	<code>Results</code>	Sorts steps by requirement order (mandatory → conditional → recommended), shows the aerial thumbnail, parcel lot facts, per-step download/view status, and the "send to Evidence Locker" selection set
<code>components/Row.js</code>	<code>Row</code>	One document row: status dot, files, verdict chips, in-locker badges, view/download/manual-upload buttons
<code>components/documents.js</code>	<code>Access</code> , ...	The step's via-link access block (calls <code>jobFileUrl</code> to view/download) and uploaded-file chips
<code>components/DocViewer.js</code>	<code>DocViewer</code>	In-app viewer: loads the file as a blob, renders PDF/HTML/image/"other", records the surveyor's match-mismatch verdict + note via <code>/api/feedback</code>
<code>components/jobs.js</code>	<code>Jobs</code> , <code>JobDetail</code>	Jobs list (<code>/api/jobs</code>) + detail (<code>/api/jobs/detail</code>) incl. manifest summary and per-doc file download
<code>components/locker.js</code>	<code>EvidenceLocker</code>	Order-scoped Evidence Locker: list <code>/api/evidence</code> , detail <code>/api/evidence/detail</code> , delete item <code>/api/evidence/item</code> , delete order, serve file <code>/api/evidence/file</code>
<code>components/about.js</code>	<code>About</code> , <code>StateTracker</code> , <code>ConnectionCheck</code>	Coverage tracker (live from <code>/api/reference/coverage</code>), release notes, and the Connection Check that probes <code>/api/linkcheck?portals=1</code>
<code>components/primitives.js</code>	<code>Logo</code> , <code>Brand</code>	G-Source hexagon mark + sidebar brand block
<code>styles/app.css</code>	—	Brand tokens <code>--gs:#33409E</code> , <code>--gs-d:#242a63</code> , <code>--gs-red:#E4482B</code> , scrollbar/skeleton/spin/fade utilities (16 lines)

2. QuickPlot (order-centric) — `/quickplot`

The shell — `frontend/quickplot.html` + `frontend/src/qp/app.js`

`quickplot.html` loads React + Babel only — **Tailwind is deliberately absent** — and ships its own design system (`frontend/src/qp/quickplot.css`). Load order: `ui.js` (primitives + API client) → `modals.js` → `research.js` → `orders.js` → `app.js` (`quickplot.html:20-24`).

`App()` (`frontend/src/qp/app.js:50`) is the shell: header (logo, tenant chip from `/api/v2/meta` , notifications, avatar), sidebar (`Users / Orders / Support / Quote hub`), and a **hash router**:

```
function parseHash() {
  // qp/app.js:40
  const h = (location.hash || "#/orders").replace(/^#\//, "");
```

```
const parts = h.split("/").filter(Boolean);
if (parts[0] === "orders" && parts[1] && parts[2] === "research")
  return { view: "research", id: parts[1] };
if (parts[0] === "orders" && parts[1]) return { view: "order", id: parts[1] };
return { view: parts[0] };
}
```

So `#/orders`, `#/orders/:id`, `#/orders/:id/research` are the three real screens; `users`, `support`, `quote` render `Placeholder` ("lives in the main Mapperty app") — a deliberate slice so the hub can be reviewed in its real surroundings and later **embedded with Mapperty's own nav** (`app.js:29-37`). A `hashchange` listener re-parses + scrolls to top (`app.js:54-59`).

The API client — `frontend/src/qp/ui.js`

One function knows the auth headers — everything else in the UI just calls it (`ui.js:14`):

```
async function api(path, opts = {}) {
  const headers = { "X-Org-Id": QP.org, "X-Actor": QP.actor, ...(opts.headers || {}) };
  if (opts.json !== undefined) { headers["Content-Type"] = "application/json";
    opts.body = JSON.stringify(opts.json); delete opts.json; }
  const r = await fetch("/api/v2" + path, { ...opts, headers });
  ...
  if (!r.ok) throw new Error(data.detail || data.message || `HTTP ${r.status}`);
  return data;
}
```

`QP` (`ui.js:9`) reads `org` and `actor` from `localStorage` (`qp.org`, `qp.actor`) with `"default"` / `"Owen Ranford"` defaults — this is the single place auth tokens replace them. `fileUrl(orderId, docId, download)` builds the `/api/v2/orders/{id}/documents/{id}/file` endpoint (`ui.js:29`).

The file also carries: formatters (`fmtBytes/fmtDate/fmtTime/fmtStamp/fmtNum/initials`), `x/fileKind` utilities, 35 SVG icons (`ic.*`), and the shared components —

Component	Purpose
<code>Btn</code> , <code>Chip</code> , <code>Spinner</code> , <code>Empty</code>	Base building blocks tuned to the token palette
<code>Modal</code> , <code>Drawer</code>	Overlays (Escape-to-close, scrim-click close)
<code>Toasts</code> + <code>toast()</code>	Global side-channel, <code>_toastBus</code> — the only global mutable
<code>useAsync(fn, deps)</code>	Load-spinner-error state machine used by every data screen
<code>MapThumb</code>	Free keyless Esri World Imagery thumbnail around <code>lat/lon</code>
<code>SearchInput</code> , <code>useDebounce</code>	Debounce search box (300 ms, one request per phrase)

The screens — `frontend/src/qp/orders.js`

- `OrdersList` (`orders.js:428`) — table of orders with live search (`/orders?q=...`, debounced), stage + research chips, locked/document counts, and the "Seed sample order" button calling `POST /demo/seed` when the list is empty (`orders.js:438`). "New Order" opens `OrderFormModal`.
- `OrderFormModal` (`orders.js:137`) — 14 `ORDER_FIELDS` + client + access-contact blocks; `validateOrder` (`orders.js:117`) mirrors the **server** rules (needs title/number/address/parcel; due ≥ received; valid email; 2-letter state) so typos surface before a round trip. `POST /orders` or `PATCH /orders/{id}`.
- `OrderDetail` (`orders.js:246`) — header, `ResearchBanner` (the single CTA, wording tracks `research_state`: `not_started` → "Start Research" → `in_progress` → "Resume research" → submitted banner, `orders.js:37-72`),

StageRail (orders.js:9, six stages created→placed→research→field_survey→cad_drafting→delivered), property card, and two tabs: **Order Information** and **Evidence locker** (downloads via `fileUrl(..., true)`). "Locker Log" / "Order Log" open `LogModal`.

- `ResearchBanner.start` routes to the research hub, calling `POST /research/start` first if the order hasn't started (orders.js:263-270).

The research hub — `frontend/src/qp/research.js`

The "Gather the research set" screen (`ResearchHub`, `research.js:168`):

- **Initial load** (`research.js:201-219`): `Promise.all` of `/orders/{id}`, `/doc-types`, `/orders/{id}/research/status`, then docs + sources in parallel.
- **Polling** (`research.js:222-240`): only while `run.state === "running"`, every **1800 ms**, hits `/research/status`; on completion reloads docs + sources + order and toasts finished/error. The interval lives in `pollRef` and is cleaned up when not running.
- **Fetch actions**: `fetchOne(key) → POST /orders/{id}/sources/{key}/fetch`; `fetchAll() → POST /orders/{id}/sources/fetch-all` (`research.js:242-253`).
- **SourceRail** (`research.js:48`) shows the source registry with per-source state chips (`idle/fetching/fetched/failed/unavailable → "In locker" / "Auto fetch failed" / "Link only"`, `research.js:5-11`), the "Open source" link, and Retry / Auto fetch buttons ("Auto-Fetch All" disabled while a run is in flight).
- **DocRow** (`research.js:87`) — per document: type picker (locked ones are read-only), status chip, download/re-fetch/delete actions, and **Review & lock** (disabled until a `doc_type` is chosen). Optimistic `doc_type PATCH` (`research.js:254-258`) with rollback reload on error.
- **RunningPanel** (`research.js:142`) — the empty-state animation during a run: the server's phased label list + progress bar (`run.percent`).
- Sticky footer (`research.js:402`): Save & Exit / Submit Research gates.

The dialogs — `frontend/src/qp/modals.js`

- **ReviewDocModal** (`modals.js:16`) — the defensive lock: three confirmation checks (`legible`, `matches_parcel`, `source_recorded`; `CONFIRMS`, `modals.js:7`) must ALL be on before "Approve & Lock" enables, then a second confirm modal ("Locking is permanent.") before `POST /documents/{id}/lock` fires with the three booleans (`modals.js:28-41`). The viewer renders images directly or an `<iframe>` for PDF/html.
- **UnlockModal** (`modals.js:127`) — reason required (recorded in the change log) → `POST /documents/{id}/unlock`.
- **DeleteDocModal** (`modals.js:166`) — `DELETE /documents/{id}?reason=...`.
- **UploadModal** (`modals.js:205`) — drag-and-drop; **one request per file** (so a single rejected file doesn't lose a batch), each row carrying its `doc_type`; `POST /documents` with a `FormData` `files` + `doc_type`.
- **SubmitModal** (`modals.js:301`) — fetches `/research/summary`, shows locked/document/ review counts, hard-blocks submit when **zero locked** (`modals.js:325`); `POST /research/submit` with an optional note.
- **LogModal** (`modals.js:392`) — `/orders/{id}/log?scope=locker|order` rendered per action type with icons via `LOG_ICON` (`modals.js:360`).
- **ChecklistDrawer** (`modals.js:409`) — `/orders/{id}/checklist`; a row ticks only when a **locked** document of that type exists (the checklist-on-locked rule is enforced here in the UI and again in the API).

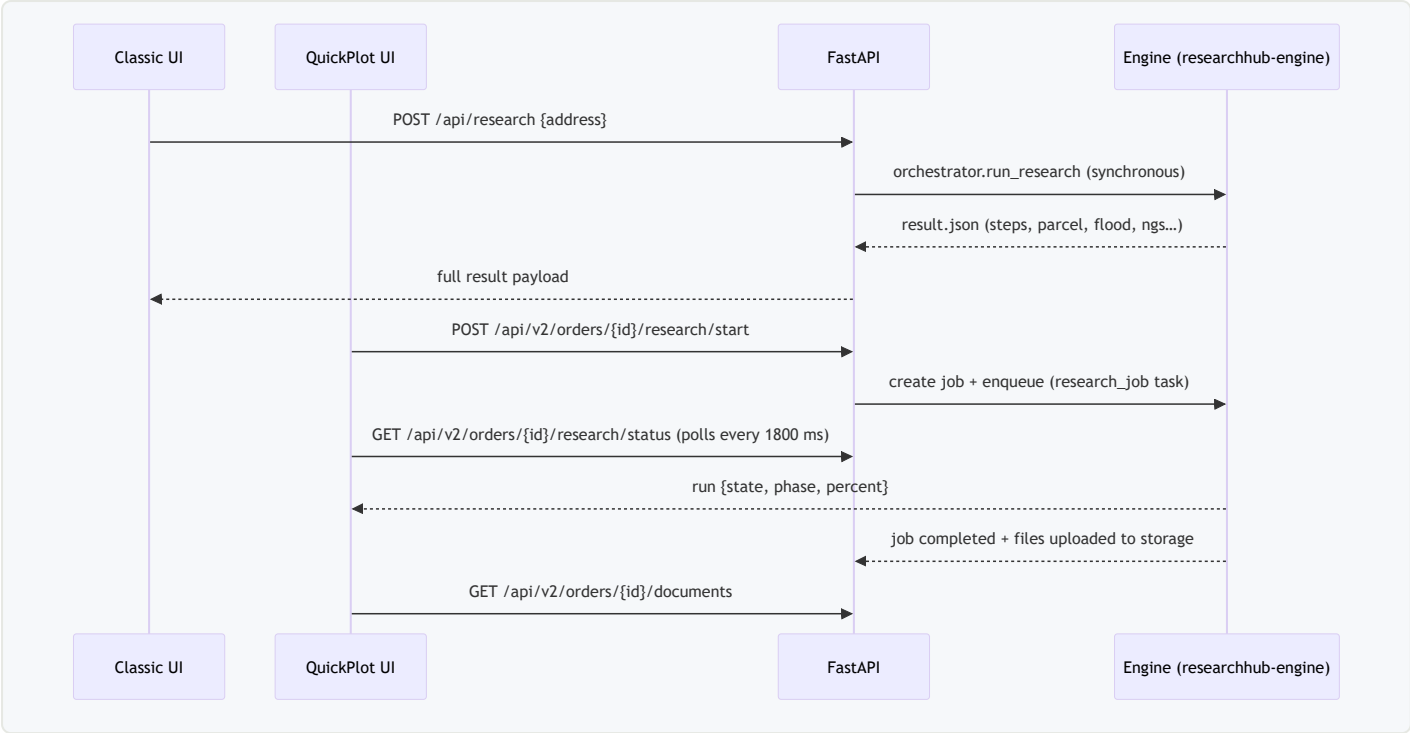
The design system — `frontend/src/qp/quickplot.css`

516 lines of plain CSS generated from the **Mapperty Figma token export** (file header, `quickplot.css:1-4`). The `:root` block (`quickplot.css:6-54`) is the contract every component uses:

- Brand ramp `--b-600: #3FA424` (primary action), `--b-500:#61D32F`, gradient `--b-grad`; neutrals `--g-*`; semantic `--warn/--danger/--info`; radii **8 / 12 / 16 px** (`--radius/-lg/-xl`); shadows `--shadow-xs/sm/lg`; layout `--header-h:62px`, `--sidebar-w:245px`; font **Figtree** (`--font`).

The `--b-*` / `--g-*` names are referenced directly in JSX (`style={{ color: "var(--b-600)" }}`), so tokens are SWAPPABLE per-tenant — a white-label build only changes this file. Animations `qp-spin/fade/slide/pop` and the `scroll` scrollbar underpin the micro-UI.

3. How the two UIs meet the same engine



Classic calls the **synchronous** `orchestrator.run_research` (deep-deep-dive §8) and shows the whole result at once. QuickPlot calls the **async job API** (`/api/v2/research/*` of `researchhub-engine`), polls status, and only then lists the uploaded evidence. Same engine, two rhythms — that's the "two UIs, one process" story end to end.

4. Frontend seams at integration

Seam	Where	What changes
<code>API = ""</code> (classic) / <code>fetch("/api/v2...")</code> (QP)	<code>lib/setup.js:4</code> , <code>qp/ui.js:21</code>	stays same-origin behind Mapperty's host
<code>QP.org</code> / <code>QP.actor</code>	<code>qp/ui.js:9-12</code>	swap <code>localStorage</code> reads for the Cognito token in <code>api()</code> ; nothing else changes
Sidebar nav Users/Support/Quote	<code>qp/app.js:20-25</code>	replaced by Mapperty's own routed nav when embedded
<code>/static</code> , <code>/</code> , <code>/quickplot</code> mounts	<code>backend/app/main.py:464-484</code>	same routes serve from the parent's static bucket
Tailwind (classic only)	<code>index.html:12</code>	stays local; no CDN dependency survives integration

04 — Data Sources Registry (BA + backend team)

How the app decides **where** to look, in what order, and when to trust the result. All paths are in this repo's `backend/app/data/` + `backend/app/services/`.

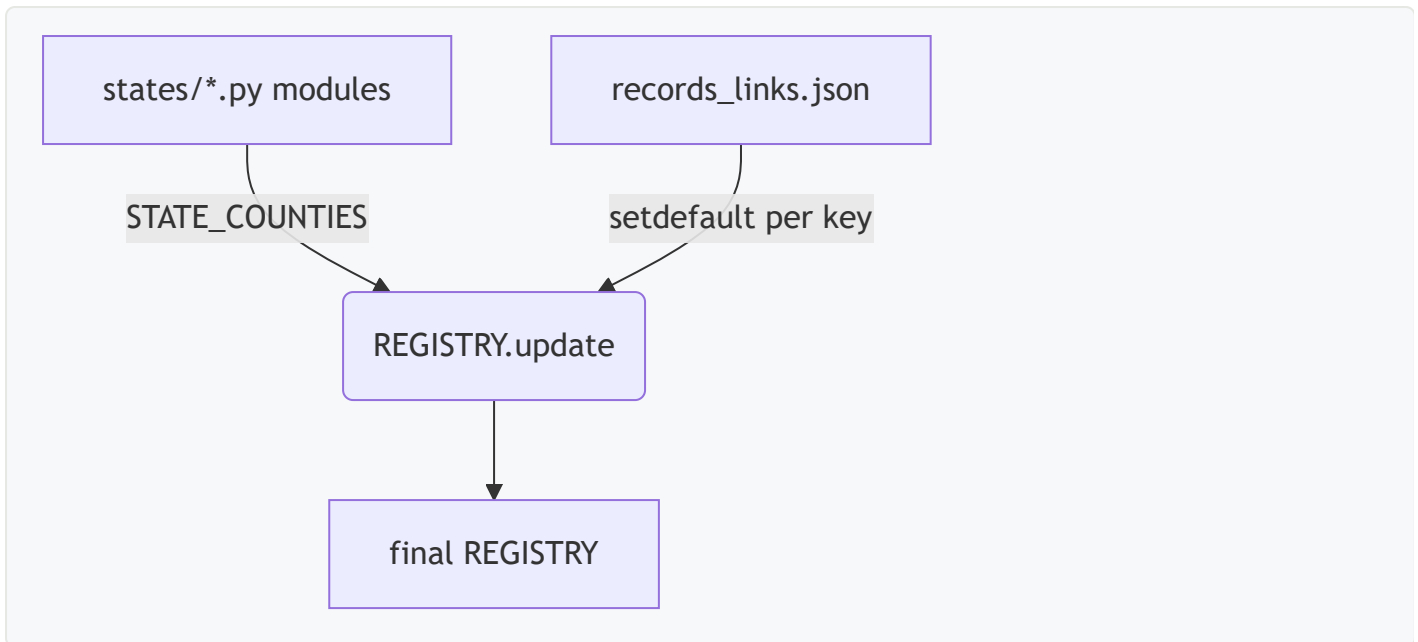
0. The five files that ARE the registry

File	Contents	Role
<code>data/county_platforms.py</code>	REGISTRY — per-county entries keyed by 5-digit FIPS (county + state), with <code>county</code> , <code>state</code> , optional <code>clerk_url</code> / <code>clerk_platform</code> , <code>appraiser_url</code> , <code>gis_rest</code> (ArcGIS MapServer), <code>parcel_layer_hint</code> , <code>appraiser_api</code> , <code>situs_exclude</code> , <code>clerk_note</code>	the runtime lookup the pipeline reads
<code>data/states/</code> package	one module per state exporting <code>STATE</code> , <code>PARCEL</code> (statewide service), <code>COUNTIES</code> (per-county overrides), <code>DEED_LINK</code> , <code>PLAT_LINK</code> , <code>APPRAISER_LINK</code>	country-scale coverage without a giant file
<code>data/records_links.json</code>	bulk-verified clerk + appraiser links for hundreds of counties (351 entries today), each carrying <code>_src_clerk_url</code> provenance	TX's 254 CADs etc. without hand-writing
<code>data/geography.py</code>	states/counties list + live Census TIGERweb city picker per county	dropdowns, FIPS normalization
<code>data/reference.py</code>	<code>SURVEY_TYPES</code> , <code>RESIDENTIAL_DOCS</code> (the 12 doc steps in declared order), <code>DOC_MATRIX</code> (which docs each survey type needs), <code>SOURCES</code>	the survey matrix

Entry example, Volusia FL (`county_platforms.py:21-28`):

```
"12127": { # Volusia
  "county": "Volusia", "state": "FL",
  "clerk_url": "https://app02.clerk.org/or_m/",      # note: /or_m/, not /or – portal moved
  "clerk_platform": "inhouse",
  "appraiser_url": "https://vcpa.vcgov.org/searches.html",
  "gis_rest": None,
},
```

1. Merge precedence (the rule that keeps curated data safe)



1. `REGISTRY` is seeded with the hand-verified FL counties (`county_platforms.py:19`).
2. Every state module's `COUNTIES` overrides are merged via `REGISTRY.update(...)` — a state-module value **replaces** a registry value for the same key (`county_platforms.py:502`).
3. `records_links.json` is merged **last**, and **only fills empty keys** — every key applied with `cur.setdefault(k, v)` (`county_platforms.py:522-528`). A curated state-module entry (richer notes, a scraper platform, a parcel service) is never clobbered by the bulk file.
4. The four aggregate dicts (`STATE_PARCEL`, `STATE_COUNTIES`, `STATE_DEED`, `STATE_PLAT`, `STATE_APPRAISER`) are built by a `pkgutil.iter_modules` loader over `data/states/` (`states/__init__.py:20-31`) — adding a state = adding one file.

`lookup(fips)` (`county_platforms.py:544`) is the only read path the pipeline uses. `netronline(state_abbr, county_name)` (`:548`) builds the always-available directory deep-link so **every county on Earth has a clerk/appraiser fallback**, even one not in the registry.

2. The six data authorities and where each is used

1) US Census Geocoder — address → lat/lon

`services/geocode.py`; endpoint `https://geocoding.geo.census.gov/geocoder/` (part of `_CORE_ENDPOINTS`, `backend/app/main.py:393`). ArcGIS + Nominatim fallbacks. The geocoder point is what the parcel step re-buffers — **it lands on street centerlines and can miss the parcel polygon by 100–200 m** (observed 92 m and ~185 m in tests, going onto the neighbor). See "Trust only on an address match" below.

2) Census TIGERweb — states / counties / cities

`services` = none of the engines'; it powers the dropdowns:

- states + counties: `data/geography.py` over the bundled `_counties_raw.json` (16 288 rows, nationwide) and `state_fips()/counties()/county_name()`.
- cities: **live** `cities(county_fips)` queries TIGERweb Places layers 4 (Incorporated) + 5 (Census Designated Places) — `geography.py:18-22`, `:120` — so the City dropdown is always current, per county, nationwide (cached).

3) Parcel services — county `gis_rest` → statewide → appraiser link

Traced in `services/parcel.py`:

1. **County** `gis_rest` (an ArcGIS MapServer; query runs through the shared session, POST for geometry-heavy queries via `http.post_json`, `services/http.py:25`).

2. **STATE_PARCEL** — a statewide FeatureServer, e.g. TX StratMap (`data/states/tx.py` , `PARCEL` block). **Do not "fix" the TX URL**: it still says `2019_Texas_Parcel_StratMap` but serves the **2025** vintage, 14.3 M parcels.
3. **Appraiser / NETROnline** link only — the "one click away" fallback for the ~1,062 counties with no queryable parcel service.

Per-county overrides that matter: `situs_exclude` and `compose_situs` (`parcel.py:153-157`). When a CAMA layer carries both situs and owner mailing address, `situs_exclude` names the mailing fields so the owner's mailbox address never leaks into the composed situs used for the address match.

Trust only on an address match. `_candidates()` (`parcel.py:331-355`) widens the query box in steps (`75 m` → `150 m` → `300 m`) and stops at the **first** radius whose parcel set contains a situs that matches the searched address (`_addr_match` , `parcel.py:215`); `_finish` flags `buffered_match: true` + `address_match` so a mis-geocode landing on a neighbor still resolves the true parcel, while a *cross-street* miss is tagged "verify" instead of silently adopting a wrong polygon (`parcel.py:431-432`).

4) FEMA NFHL — flood zone + map

`services/fema.py` against `hazards.fema.gov/arcgis/rest/services/public/NFHL/MapServer` (`core, main.py:395`). The FloodAdapter (`02 §8`) accepts the zone JSON **or** a composited map exhibit — `services/downloader.py` renders the panel with **Pillow** and saves the PNG so the evidence has a human-readable map, not just a JSON code. Sandbox note: this network blocks `hazards.fema.gov` (TLS reset) — test the fallback-link path from here, not the fetch.

5) NOAA NGS — survey control / benchmarks

`services/ngs.py` against `geodesy.noaa.gov` (`core, main.py:397`). Radial search around the lot; up to 3 datasheets downloaded (`NGS_datasheet_AK1659.html` is the E2E artifact). Returns `empty` (not `error`) when no marks exist nearby — "none found" is a valid research answer.

6) County Clerk / Recorder — deeds, plats

`services/clerk_scraper.py` — **Playwright** against county portals; scraper adapters per platform:

- **NewVision BrowserView** (NJ Middlesex/Morris/Cape May), **AcclaimWeb** (NJ Somerset), **Tyler/Eagle** (FL Manatee/Volusia families), Kofile **PublicSearch** (`<county>.tx.publicsearch.us` , 53 TX counties), Kofile **County Fusion** (`countyfusion<N>.kofiletech.us/...?countyname=...` , TX + OH). All in `_scrape_documents` — **one browser launch per order**, cached on `ctx._clerk_docs` , so deed and plat steps never open the browser twice.
- `appraiser.py` — Polk CAMA record card download; everywhere else the tax card is generated from the parcel records, with a VERIFIED appraiser portal link (Georgia: `qpublic.net/ga/<slug>/`); Texas: Comptroller county directory per county.
- Anything on an `inhouse / unknown` platform with no scraper → the `_attempt_clerk_adapter` (`sources.py:438`) tries the shared clerk scrape and falls back to the verified clerk `link` (never fabricates one).

3. The coverage gates (what "adding a county" means)

`/api/reference/coverage` (`main.py:247-388`) computes everything **live from the registry**, with two edge-case corrections built in:

- **id_only statewide layers don't count** (`main.py:294-296`): Ohio's ODNR layer answers Parcel-ID lookups but carries no situs, so it can't auto-fetch from an address — the app's real entry point. Treating it as "statewide" would credit OH all 88 counties when only 33 resolve. Expect the same caveat on any `id_only` service you add.
- **Partial statewide** (`counties_covered` , `main.py:300`): TN's layer covers 86/95 counties, CO's ~38/64. The gate caps the credit at the declared count.

Adding a county parcel service — verify COVERAGE, not reachability. A layer that *loads* is not enough. A found candidate layer must:

1. return a parcel for **≥60% of a grid of points inside the county polygon** (a bbox or single point check passes city-only / study-area look-alikes — Walton GA's reachable copy covers 8%, Ware GA's parcels×soils intersect has 3× the true row count);
2. expose a **parcel-id field**; and
3. be checked for **situs vs mailing** fields, setting `situs_exclude` on CAMA layers carrying both.

This is the single most-common failure mode in the regional GIS orgs' catalogs — the documented gate is the guard. (Harness support: `scripts/discover_state_parcel.py`, `scripts/verify_situs_fields.py`, `scripts/audit_parcel_candidates.py`.)

4. Link verification + Connection Check

`services/http.py:33` `check_url` returns one of four verdicts — the Connection Check surface (`/api/linkcheck`, `main.py:402-432`) reflects the **user's own** network + K7:

Verdict	HTTP evidence	Meaning for the user
ok	status < 400	reachable, expected
blocked	401 / 403 / 406 / 429	bot/geo WAF block — opens fine in a browser ; don't treat as dead
broken	404 / 410 / 5xx	URL/path wrong or server error — fix the URL (portals move: Volusia <code>/or/</code> → <code>/or_m/</code>)
offline	no HTTP response (TLS reset / timeout)	AV/firewall/Egress blocking — the K7 case or a sandbox VPN

The shared `session` (`http.py:8-16`) carries the app's `User-Agent`, retries 429/500/502/503/504 with backoff, and is reused by the fast two-click `check_url` (`allow_redirects=True`, `stream=True`).

5. The survey matrix (`data/reference.py`)

`RESIDENTIAL_DOCS` (from `POI = apartment/office... families`) is the agreed order the engine runs: the 12 steps (parcel, appraiser, **deed**, **plat**, **adjoiners**, **easements**, **prior_survey**, **condo**, flood, benchmarks/ngs, glo, zoning) with their `requirement` (mandatory → conditional → recommended). `DOC_MATRIX` maps `survey_type` → the subset of docs that survey needs, and `SURVEY_TYPES` lists the supported types — a new survey type = a matrix row, nothing else.

6. Live numbers to quote (recompute command: `hit /api/reference/coverage`)

- National parcel auto-fetch: **2,082 / 3,144 counties = 66.2%** (and higher order-weighted).
- FL **51.7%** of order volume: parcels 67/67, deed+plat 67/67, appraiser 67/67.
- GA **33.8%**: parcels 90/159 (Schneider/qPublic HTML counties need the paid API — deeds/plats and appraiser are 100% via GSCCCA + qpublic.net).
- TX **7.1%**: parcels 254/254 (StratMap), clerk per-CAD directory, appraiser 254/254.
- OH **1.5%** (33/88 situs-resolvable), NJ **1.1%** (21/21 all three).

05 — Deployment & US Egress (DevOps + backend team)

Every way this system runs, and the one question that used to consume whole meetings — **"how do the US-only government APIs work?"** — answered from the code.

0. One application, three run modes

Mode	What runs	How to start
Local laptop (fastest)	uvicorn + SQLite + in-process orchestration	<code>../.venv/Scripts/python.exe -m uvicorn app.main:app --port 8000 --reload --reload-dir app</code> (or <code>run.bat</code>)
Docker stack (full engine)	Postgres + Redis + uvicorn + Celery worker	<code>docker compose up -d --build</code>
E2E harness (real everything)	real Postgres + Redis + uvicorn + real worker, driven over HTTP	<code>../.venv/Scripts/python.exe scripts/e2e_local.py</code>
Production (parent)	AWS us-east-1 ECS: quickplot-dev-api + quickplot-dev-worker, Cognito, alembic as one-off task	parent <code>deploy-dev.yml</code> workflow → ECR → ECS

Local quick-start details

- `RUN_ENV=local` shortcuts migrations — the app does `Base.metadata.create_all` at startup (`backend/app/main.py`), so a laptop never needs alembic. `RUN_ENV=test` is the same path for tests.
- uvicorn `--reload --reload-dir app` watches `.py`; the frontend is served **no-cache** by middleware (`main.py:35`), so JS/HTML edits appear on a plain browser refresh.
- Headless mode: `run-hidden.vbs` (output → `server.log`) + `stop.bat` (kills port owner and the whole process tree — with `--reload` there is a watcher **and** a multiprocessing worker).

1. The Docker stack (engine `docker-compose.yml`)

Four services, health-gated:

```
Browser → api (uvicorn, :8000) → db (postgres:15, named volume `researchhub_pgdata`)
      |
      |→ redis:7-alpine (/0 broker for Celery)
      |→ worker (celery -A app.engine.worker.celery_app --queues=research)
```

- Dockerfile: `python:3.13-slim`, `PYTHONPATH=/app/backend`, default CMD `uvicorn app.main:app --host 0.0.0.0 --port 8000`.
- `api` mounts `./backend` as a volume (hot code + `--reload` in dev); `worker` mounts the same backend and runs the Celery app with `--queues=research` to match the broker `redis://redis:6379/0 / result backend /1`.
- Migrations run once, as its own step: `alembic upgrade head` (config: `alembic.ini` → `script_location = backend/alembic`).
- `.env.example` documents every knob: `DATABASE_URL`, `REDIS_URL` / `CELERY_*`, `S3` (`AWS_REGION=us-east-1`, `S3_ARTIFACTS_BUCKET`), `CORS_ORIGINS`, `MAX_RETRIES_PER_DOCUMENT=3`, `DOCUMENT_TIMEOUT_SECONDS=60`.

Windows worker note — `--pool=solo`

Celery's default billiard *process* pool raises WinError 6/5 on Windows. Locally the worker must run `celery ... worker --pool=solo` (the E2E does this automatically). Linux prod uses the default prefork.

2. The E2E harness (`scripts/e2e_local.py`)

The proof that the whole contract works on **real infra** — not mocks:

1. picks a free API port (`e2e_local.py:42`), starts uvicorn and a **real** Celery worker (`--pool=solo`) with an env pointing at Postgres on port **5433** and the stock Redis;
2. runs `alembic upgrade head` on Postgres (proving the full `0002→0001→0003→0004` chain);
3. seeds a tenant + order via the dev `orders` stand-in;
4. `POST /api/v1/research/jobs` → worker consumes over Redis → polling `GET /jobs/{id}` until terminal;
5. reports per-document statuses and **verifies the blob artifacts on disk** (sha256 + size match the DB row).

Port 5433 vs the compose default 5432 is the `dc.e2e.yml` override from the sandbox shell (Native Postgres on 5432 had an unknown password; the Docker `db` runs on 5433 for E2E). The sandbox's expected degradations (FEMA/clerk TLS-reset/403) are deliberate — each source falls back to a link/retryable error, the job reaches a terminal state, and the evidence set is still defensible. Last run: job `ed46501f...` → `completed`, 6 fetched / 1 uploaded / 0 failed.

3. Production — AWS us-east-1 (from `mapperty-reference`'s `deploy-dev.yml`)

The parent deploys the merged stack to **AWS us-east-1**:

- **ECR** `quickplot-api` (image push, prod account) → **ECS cluster** `quickplot-dev` with two services — `quickplot-dev-api` and `quickplot-dev-worker`; API stores are: `APP_SECRET_KEY`, `COGNITO_CLIENT_SECRET`, `STRIPE_SECRET_KEY` in Secrets Manager (`deploy-dev.yml:15-28`).
- **Cognito** user pool `us-east-1_EHikamkop` guards the API (the auth headers the engine's `dependencies.py` placeholder will replace from).
- `alembic upgrade head` **runs as a one-off ECS task inside the VPC** (`deploy-dev.yml:190-265`) — the migration chain ships with the image, then the service comes up.

This is the doc-defining point: **a US-region host satisfies FEMA/county geo-blocks natively**. Outbound calls originate from a US IP automatically; no VPN, no fixed IP, no per-machine setup (`poc01/SurveyResearch/docs/DOCKER_AWS_DEPLOY.md:17-21`).

4. US egress — the decision, honestly documented

Production: native. See §3.

Dev on a personal PC: the developer's PC needed a fixed **US VPN IP** because FEMA and many appraiser/clerk endpoints geo-block non-US IPs. That was the POC workaround, not a product requirement. Supporting files in the POC repo:

- `docker-compose.vpn.yml` — WireGuard override (provider-agnostic; gluetun sidecar routes **only** the outbound research calls through a US WireGuard endpoint, leaves your LAN alone). Set from any provider's `.conf`:
`WIREGUARD_PRIVATE_KEY/ADDRESSES/PUBLIC_KEY/ENDPOINT_IP/ ENDPOINT_PORT`.
- `docker-compose.proxy.yml` — the same, via an HTTP forward proxy.
- `run-on-vpnbox.bat`, `reverse-tunnel.bat`, `vpn-proxy.bat`, `docs/vpn-proxy-setup.md` — the VPN-box script lives to tunnel the laptop's traffic until the AWS deploy existed.

Why this matters for engineers reading this briefing: do NOT re-introduce a VPN dependency into the engine. The contract should treat egress as an infrastructure concern. If a future tenant is non-US-hosted, the WireGuard/proxy overrides compose in at the infra layer — the engine's callers keep pointing at the plain session in `services/http.py`.

5. Remaining real-world risks (what *still* needs engineering, not infrastructure)

1. **Shared-IP WAF/rate limiting** — QA finding **L-5**. A cloud region IP is shared by many users; clerk/appraiser portals WAF-block datacenter IP ranges. Expected symptoms: 403 on `check_url` ("opens in browser") and throttled Playwright scrapes. Mitigations are app-level (per-source cool-downs, backoff, honoring `Retry-After`), not re-deployments.
2. **K7-style AV on the operator's machine** blocks FEMA + some clerk/appraiser domains with a TLS reset (verdict `offline`). That's an AV allow-list fix, not a code fix — Connection Check exists precisely to surface it.

3. **Portals move** — verified URLs rot (Volusia `/or/` → `/or_m/`; Hillsborough/Lake were stale). `check_url` reports broken; the registry + `records_links.json` re-verification scripts (`scripts/verify_links.py`, `scripts/verify_county_portals.py`) are the sweep.

6. Integration checklist (deployment shape that makes all of this work)

- Static frontend mounts (`/static`, `/`, `/quickplot`) — no build step, ship the same folders.
- Alembic: drop `0002_dev_base_tables.py`, retarget `0001`'s `down_revision` onto the parent chain; run `upgrade head` as the one-off (already the parent's pattern).
- Worker partition: `--queues=research` matches `enqueue_research_job`; `task_acks_late` + the retry policy requeue on crash.
- Storage: keep `build_key()`'s "reject falsy document_id" guard; choose `local` or `s3` via `QP_STORAGE_BACKEND` once per deployment.

06 — Glossary & Conventions (every team, quick reference)

The words, variable names, and rules everyone must use the same way. This is the single dictionary; if a term isn't here, it was probably renamed or doesn't exist.

1. Status vVocabulary (FROZEN — never change meaning)

Document step statuses (what the UI renders) — `engine/contracts.py:20` `StepStatus`

Value	Meaning front-to-back
<code>ok</code>	a real artifact was fetched/saved for the step
<code>link</code>	no auto-fetch; a working records link is provided as the fallback
<code>empty</code>	queried for real, found nothing (valid answer — e.g. no NGS marks nearby)
<code>error</code>	the step failed and needs review/retry

Engine-only outcomes (why a status is what it is) — `contracts.py:28` `SourceOutcome`

`auto` · `link_only` · `blocked` (401/403/406/429 → WAF/browser-fine) · `broken` (404/410/5xx → URL wrong) · `retryable` (timeout/connection) · `manual_review`.

Job lifecycle — `engine/schemas.py` `ResearchJobStatus`

`queued` → `running` → { `completed` | `partial` | `failed` | `cancelled` } (+ `cancelling`); terminal rules: all ok → `completed`, some failed → `partial`, none ok → `failed` (`engine/service.py:99`).

Per-document job statuses — `ResearchDocStatus`

`queued` → `fetching` → `fetches` → `uploading` → `uploaded` → (`failed` | `skipped`).

Confidence — `contracts.py:51`

`high` (address-matched parcel) · `medium` (deed/plat from a clerk scrape) · `low` · `none`.

2. Document keys (one canonical key per document; legacy aliases map onto them)

Engine <code>doc_type</code>	Classic step key	UI label
<code>PARCEL_RECORD</code>	<code>parcel</code>	Parcel / Parcel ID
<code>PROPERTY_APPRAISER_TAX_RECORD</code>	<code>appraiser</code>	Appraiser / tax card
<code>DEED_SUBJECT_PARCEL</code>	<code>deed</code>	Deed (current vesting)
<code>RECORDED_PLAT_SUBDIVISION_MAP</code>	<code>plat</code>	Plat / subdivision map
<code>FEMA_FLOOD_ZONE_FIRM</code>	<code>flood</code>	FEMA flood map + zone
<code>NGS_CONTROL</code>	<code>benchmarks</code> / <code>ngs</code>	Survey control / benchmarks
—	<code>adjoiners</code> <code>easements</code> <code>prior_survey</code> <code>condo</code>	link-only clerk docs
—	<code>glo</code>	BLM GLO (always link)

Alias normalization: `"flood"` → `FEMA_FLOOD_ZONE_FIRM` (any casing), step short-names → canonical (`engine/service.py:248`). **Rejected doc types are a 422**, never a silent skip.

3. The QuickPlot lockdown variables (server-sided on purpose)

- `research_state`: `not_started` → `in_progress` → `submitted` (order moves to **field_survey**).
- Lock requires `doc_type` **and all three** confirmations `legible` + `matches_parcel` + `source_recorded` (`quickplot/modals.js:7`, enforced again in `router.py`).

- Locked documents: cannot be re-typed, re-fetched or deleted. **Deletes are soft** — the audit log keeps pointing at a real row.
- **Submit is blocked at 0 locked documents** (the lock-seal is the evidence gate).
- `X-Org-Id` / `X-Actor` headers: every `/api/v2` query is `org_id`-scoped; `org_of()` / `actor_of()` are the two functions to replace when real auth lands.

4. Geocoding / parcel rules

- **Address is the only required field.** State/County are dropdowns; the *selected* county wins over the geocoded one, with a mismatch warning (multi-county edge case).
- Parcel chain: county `gis_rest` (MapServer) → `STATE_PARCEL` (FeatureServer) → appraiser / NETROnline link.
- Point misses widen by `75 m` → `150 m` → `300 m` (`services/parcel.py:349`) and stop at the first radius with an address match; a buffered-but-unmatched hit is `buffered_match` + **verify**, never silently adopted.
- `situs_exclude` / `compose_situs` (`parcel.py:153-157`): per-county overrides so the owner **mailing** address never leaks into the situs used for matching.
- TxGIO StratMap: URL says `2019_...` but serves 2025 data — **do not "fix" the URL**.
- Coverage gate for new parcel services: $\geq 60\%$ of a county-point grid, ≥ 400 features, a parcel-id field, and a situs-vs-mailing check (`situs_exclude` set when both present).

5. Storage & keys

- **Never build blob paths below** `services/storage.py`. `build_key(org, order, doc_id, fn)` **raises** on a falsy/ "None" `document_id` (`storage.py:42-43`) — that guard is what fixes the "every document shared one blob" regression.
- `QP_STORAGE_BACKEND=local|s3`; a broken S3 config falls back to local with a log line — bad config never takes the app down.
- Scratch folders (`jobs/`, `data/`) hold staging artifacts + `manifest.json` / `result.json`; the blob store is the record of record.

6. Registry & data rules

- Merge order: hand-verified FL seed → `STATE_COUNTIES` `REGISTRY.update` → `records_links.json` with **setdefault**, **so curated state-module entries always win** (`data/county_platforms.py:502,522-528`).
- Add a state = add one file under `data/states/` exporting any of `STATE/PARCEL/COUNTIES/DEED_LINK/PLAT_LINK/APPRaiser_LINK`; the pkgutil loader aggregates it.
- `id_only` statewide layers don't count as "auto-fetch from address" coverage (OH ODNR).
- **Never invent a records URL.** Prefer `.gov/.org/.us`; verify live before merging.
- Every document step carries a `link` fallback (`ok|link|empty|error`); adopters raise `SourceError`, steps classify, the job still completes.

7. Egress & connectivity verdicts (`services/http.py:33` `check_url`)

- `ok` (<400) · `blocked` (401/403/406/429 — browser-fine) · `broken` (404/410/5xx — fix the URL) · `offline` (TLS reset/timeout — AV/firewall, the K7 case).
- 403 in `/api/linkcheck` = **"opens in browser"** — do not report the site as dead.

8. Deployment vocabulary

- `RUN_ENV=local|test` → `create_all` at startup (no migrations); prod runs `alembic upgrade head` (engine chain `0002` shim → `0001` → `0003` → `0004`; parent drops `0002`).
- Producer→worker contract: `ack_late`, `worker_prefetch_multiplier=1`, `--queues=research`, Windows `--pool=solo`.
- US egress: native on AWS `us-east-1` (parent ECS `quickplot-dev`); dev-PC VPN (WireGuard or HTTP proxy overrides) is a *developer convenience*, not an engine concern.

9. Testing vocabulary

- `scripts/export_contracts.py` re-exports `contracts/` — a drift in the checked-in schemas is a contract break by definition.
- `scripts/qp_integrity.py` re-hashes stored blobs against recorded digests (`MISSING` / `MISMATCH` / `SHARED`) — run after any storage/ingest change.
- E2E green = contract green: real uvicorn + real worker + real Postgres(5433)/Redis over HTTP.

07 — Opportunities & Roadmap (BA + leadership)

What's committed, what's already decided against, and what a good engineer would flag next. Ordered by **order-volume impact per unit of effort** — the discipline the whole build-out uses.

1. The committed backlog (from `docs/HANDOFF.md` \$5, unchanged)

Order volume by state: **FL 51.7% · GA 33.8% · TX 7.1% · OH 1.5% · NJ/NC 1.1%** (source:

`Knowledge/Nationwide_Expansion_Research.xlsx`, "State Availability"). Everything below is sequenced against that, not population.

#	Opportunity	Unblocks / why	Effort shape
P1	Paid parcel/records API	GA's 69 remaining parcel counties (Schneider/qPublic HTML, no REST — exhaustively confirmed) + deed/plat document gap nationwide . Candidates: Regrid/ReportAll (parcels), DataTree/CoreLogic (deed images). ATTOM was evaluated and rejected — deed <i>data</i> , not a downloadable document.	vendor + adapter
P2	Kofile PublicSearch scraper adapter	deed/plat auto-download for 53 TX counties at once (Dallas, Tarrant, Bexar, Collin, ...) — identical app at <code><county>.tx.publicsearch.us</code> . Deed/plat auto-download is only 5 counties nationwide today; this is the biggest functional gap.	one adapter → 53 counties
P3	NJ deed/plat auto-download	Somerset = AcclaimWeb (adapter exists); Middlesex/Morris/Cape May = NewVision BrowserView (same family as Polk FL). Configuration, not new scraper work.	config-only
P4	Kofile County Fusion adapter	11 counties (OH + TX) at <code>countyfusion<N>.kofiletech.us/...</code> . Same shape as P2, smaller win.	adapter
P5	Rural link tail	TX deed 42, OH deed 16, OH appraiser 9 — already return directory links; patterns exhausted; ~67 individual searches. Recommendation: do not prioritize over P1–P4.	low value
P6	Nationwide long tail	1,027 uncovered parcel counties across 16 states (KY 118, MO 108, KS 101, ...) = <2% of orders . Best closed by P1 , not by hand.	avoid

Platform batch table (one scraper → many counties) — from `HANDOFF.md` §6:

Platform	URL shape	Where	Status
Kofile PublicSearch	<code><county>.tx.publicsearch.us</code>	53 TX	✗ P2
Kofile County Fusion	<code>countyfusion<N>.kofiletech.us/countyweb/...?countyname=...</code>	11 TX + OH	✗ P4
AcclaimWeb	<code>.../AcclaimWeb/</code>	FL ×3, NJ Somerset	✓ <code>acclaim</code>
NewVision BrowserView	<code>.../publicsearch/</code> , <code>.../searchapplication/</code>	Polk FL, NJ ×3	✓ partial
Tyler Eagle	—	FL	✓ <code>eagle</code>
Landmark	<code>.../LandmarkWeb</code>	FL ×2	✓ <code>landmark</code>

Vendor-batch probing rule (already learned the hard way): **control-test with a bogus county name first**. The County Fusion probe did this — a fake county correctly matched nothing, the only way to trust an echoed "`<County> County`" as a real hit rather than a generic login shell.

2. Already decided against (do not re-propose without new data)

- **ATTOM** for deeds — returns data, not a downloadable document.

- **Hand-wiring the TX 254 CADs / OH links** — bulk `records_links.json` regenerated by `scripts/build_records_links.py`; merge rule keeps curated entries winning.
- **"Fixing" the TxGIO StratMap URL** — 2019-named URL serves 2025 data.
- **Free-parcel harvesting for the 69 GA qPublic/Schneider HTML counties** — no queryable REST, confirmed exhaustively; only P1 closes them.

3. SaaS productionization (QuickPlot is tenant-ready, not tenant-secure)

Built `org_id`-scoped everywhere, but **auth is intentionally absent right now** — `X-Org-Id` / `X-Actor` are self-asserted (`quickplot/router.py` `org_of()` / `actor_of()`).

Item	Today	The move
Tenancy	every <code>/api/v2</code> query <code>org_id</code> -scoped; <code>qp_*</code> tables per-tenant-aware	no schema change
Auth	header placeholder (<code>engine/dependencies.py</code>)	swap <code>org_of</code> / <code>actor_of</code> for Cognito deps (endpoint signatures unchanged) + pass token in <code>qp/ui.js</code> <code>api()</code>
Worker scale-out	in-memory <code>_RUNS</code> registry in <code>research.py</code> (single-process assumption)	seam is <code>start_run()</code> / <code>run_status()</code> only → Celery/RQ
Storage	<code>local</code> FS	<code>QP_STORAGE_BACKEND=s3</code> (bucket per org or org-prefixed keys); run <code>qp_integrity.py</code> on a schedule
Evidence	standalone <code>evidence_source.py</code> shim	read the parent's evidence <code>File</code> row in <code>mappers._to_file_reference</code>
Frontend	embedded slice; Users/Support/Quote are placeholders	swap <code>qp/app.js</code> nav for Mapperty's own routed nav
Blob integrity	digest stored per upload	periodic <code>MISSING</code> / <code>MISMATCH</code> sweep is already scripted

Don't expose `/api/v2` outside a trusted network until auth is in — that's the readiness line.

4. Brainstormed — new, not yet committed

Fraction of the ideas that survive contact with the codebase; nothing here is promised.

1. **Deed auto-download everywhere we already hold the plat book/page.** The clerk scrape already extracts plat/OR book-page cues once per order (`ctx._clerk_docs` cache). The gap is portal-specific *image download* logic (PDF/JPEG capture from Acclaim/Landmark/Eagle result pages). P2+P3 are the first 60+ counties; after that the same adapter shape pays for years.
2. **FEMA zone as sited evidence.** Today zone + composited map PNG are both captured. A zip of the FIRM panel or the NFHL JSON alongside would let QA eyeball "AE / 10-ft datum" without opening external links — checks the "flood insurance may be required" checklist line.
3. **NGS marks → site-control package.** We download ≤ 3 datasheets but the field crew wants the "nearest recoverable" marks ranked by distance + datasheet period. Cheap: sort + render one control summary PDF per order.
4. **Geocode→parcel self-healing telemetry.** `buffered_match` events are *already* tagged (`parcel.py:431`). Aggregating them per geocoder would let us pin which Census/ArcGIS results need the 75/150/300 widening — and whether the widening should shrink for some states.
5. **Order-level confidence score.** Per-document `Confidence` (high/medium/low/none) exists but isn't rolled up. A locked-set "research confidence" shown at submit time would make the hand-off to field survey more honest than "all locked."
6. **Change-log diffing for re-fetched documents.** The audit log records the event but not the content diff; storing `sha256` before/after on unlock+refetch would turn "was replaced" into "here's exactly what changed."

7. **Cities dropdown warm cache.** TIGERweb is live + cached, but each new county pays a first hit; a per-county cache file in `data/` would make dropdowns instant offline.

5. Parent-integration checklist (engine → mapperty-reference)

From `docs/INTEGRATION.md` §8 and the seam tables in `02`:

1. **Data chain:** swap `order_provider` body to read `app/modules/orders`; make `_to_file_reference` read the parent's evidence `File` row; replace `dependencies.py` tenant/actor with Cognito deps.
2. **Migrations:** drop `0002_dev_base_tables.py`; retarget `0001.down_revision` onto the parent chain; keep the alembic-one-off-ECS pattern.
3. **Worker:** keep `--queues=research, ack_late, %worker_prefetch_multiplier=1`. For multi-worker, move the `_RUNS` registry out of the process (the `start_run/run_status` seam).
4. **Background research spend:** batch agent work; check `/usage-credits` before large fan-outs (an org monthly limit has already killed agents once).
5. **Evidence line:** the parent `files` table missing `order_id` is a known gap — the engine's `OrderFile` join row is the pattern to adopt.
6. **Static UI:** ship `frontend/` unchanged; `/static`, `/`, `/quickplot` route and the no-cache middleware carry over as-is.

6. The one number to measure success by

- **Today:** nationwide parcel auto-fetch **66.2% of counties (2,082/3,144)**, ~92.6% of order volume covered for parcels, deed/plat auto-download in **5 counties**.
- **P2 + P3 alone** move deed/plat auto-download from 5 → ~ 65 counties (TX 53 + NJ 4 + the same-adaptor FL/NJ overlap) — the single highest-leverage line in the roadmap after P1.